Red Hat | Packmate Workshop

Participant Guide

Self-service OpenShift AI workshop for building an agentic AI application with MCP and OGX.

> **Workshop flow:** create your project and workbench, bootstrap your sandbox, discover the shared model, prototype in Playground, enable MCP tools, inspect the code, use the Packmate app, and run the evaluation pipeline.

Rendered 2026-10-12 15:31

How to use this guide

This guide is written for a data scientist or AI developer who may be using OpenShift AI for the first time.

For each module, focus on three questions:

1. Where do I click?
2. What does this screen or action mean?
3. How does this connect to the final application?

Your role in this workshop

You are acting as a data scientist or AI developer.

Your job in this workshop is not to administer the OpenShift AI platform. Your job is to:

- discover the prepared project, workbench, and shared model
- prototype behavior in Playground
- enable prepared MCP tools
- understand how tool-assisted AI responses work
- inspect the application code in Code Server

- connect the UI experience to the backend mechanism

The platform team has already prepared the heavy infrastructure for you:

- the shared model
- the serving stack
- OGX
- the MCP servers

Your learning goal is to understand how to build with those prepared capabilities, not how to install them from scratch.

Workshop story

Packmate is the single story that connects the whole workshop.

You will progressively answer questions such as:

- What should I pack for Rome next week?
- Can I take a 150 ml liquid in cabin baggage?
- Can I put a power bank in checked baggage?

The same story will appear in four places:

- in Playground, where you prototype prompts and tool use
- in Python, where you call the model and the deployed application
- in the MCP servers, which expose live tools
- in the final Packmate web application

Module 0 - Access your sandbox and OpenShift AI

WHAT YOU WILL DO

Open the OpenShift console, keep your personal `oc login` command ready, and open Red Hat OpenShift AI.

WHY THIS MATTERS

You will need the browser console for navigation and the `oc login` command for the self-service bootstrap later in the workshop.

TIME

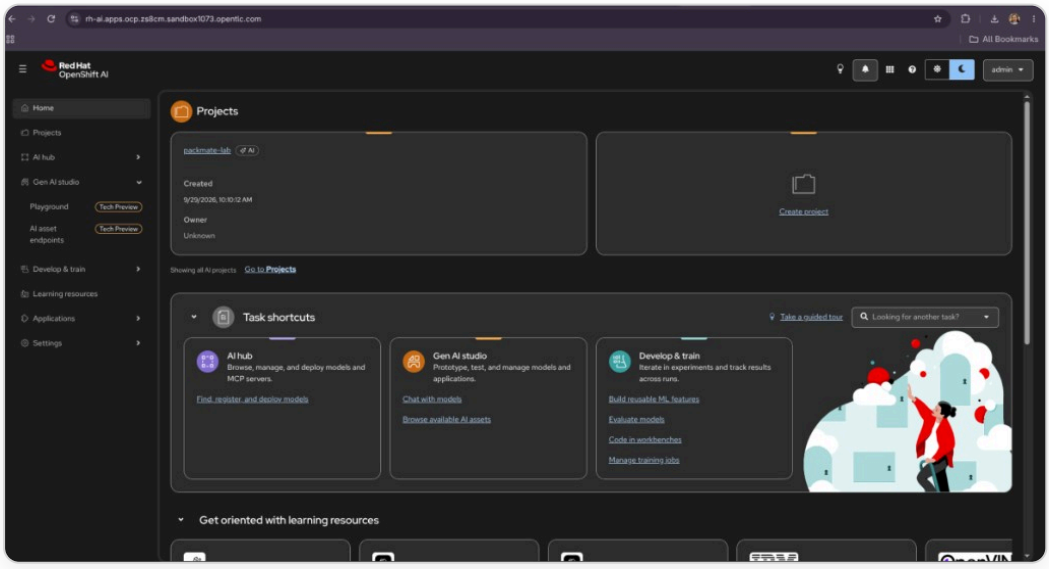
5 to 10 minutes

STEP-BY-STEP INSTRUCTIONS

1. Open the OpenShift web console URL provided with your sandbox.
2. Sign in with the sandbox credentials provided to you.
3. In the OpenShift console, open the `?` help menu and select **Command line tools**.
4. Click **Copy login command**.
5. If OpenShift asks you to authenticate again, do it, then copy the full `oc login ...` command and keep it for later.
6. Open **Red Hat OpenShift AI** from the application launcher or from the dashboard link provided by your sandbox.
7. Confirm that the OpenShift AI dashboard opens successfully.

SCREENSHOT

> **Screenshot: OpenShift AI dashboard home page**



EXPECTED RESULT

You are inside OpenShift AI and you have your own `oc login` command ready for later use.

WHAT JUST HAPPENED?

You prepared the two access paths used in this workshop:

- the web UI for OpenShift AI
- the CLI command used to bootstrap your own sandbox safely

CHECKPOINT

Before continuing, make sure you can open OpenShift AI and that you have copied your own `oc login ...` command into a temporary note.

IF IT DOESN'T WORK

Ask the instructor to confirm that your sandbox account is active and that the OpenShift AI dashboard is enabled.

Module 1 - Create your AI workspace

WHAT YOU WILL DO

Create the `packmate-lab` project and create a CPU Code Server Workbench.

WHY THIS MATTERS

The project groups your AI resources. The Code Server Workbench is your browser-based coding environment.

TIME

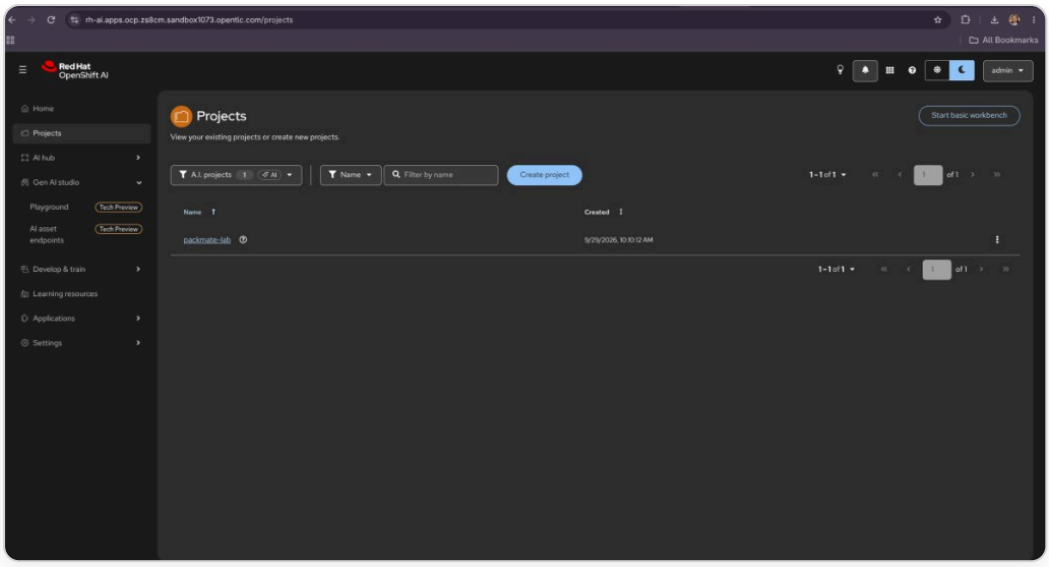
10 to 15 minutes

STEP-BY-STEP INSTRUCTIONS

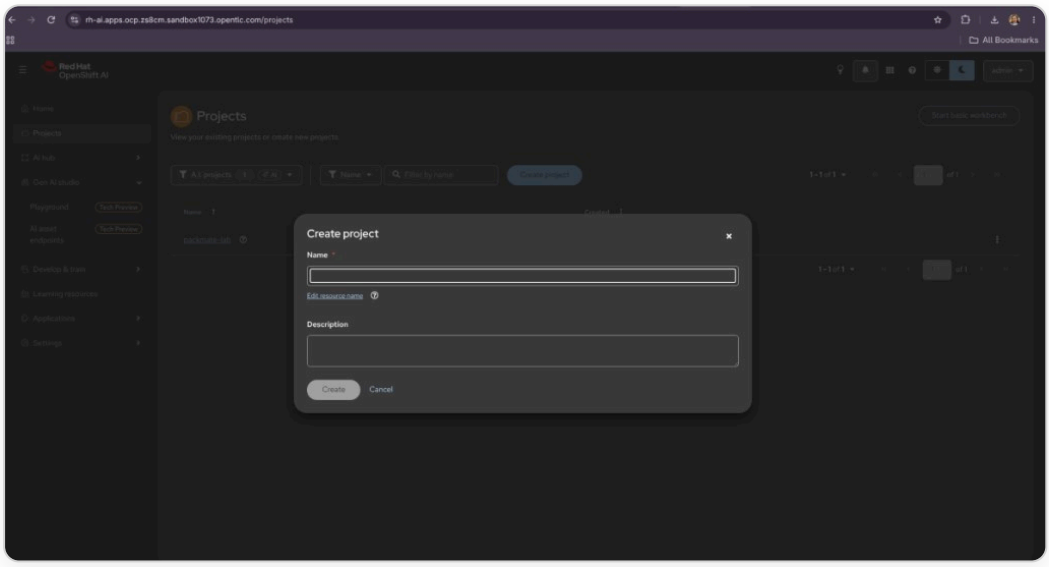
1. In the left navigation, select **Projects**.
2. Click **Create project**.
3. Enter the name `packmate-lab`.
4. Open the new project.
5. Open the **Workbenches** tab.
6. Click **Create workbench**.
7. Choose the image **Code Server | Data Science | CPU | Python 3.12**.
8. Enter the name `packmate-workbench`.
9. Create the workbench.
10. Wait until the Workbench status is **Running**.
11. Open the Workbench.

SCREENSHOTS

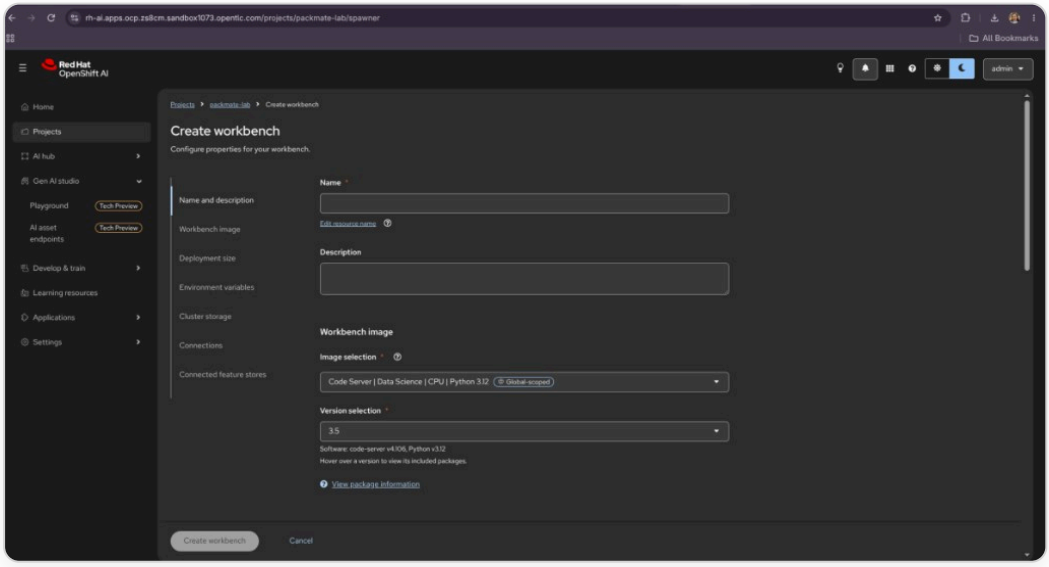
> **Screenshot: Projects page before creating `packmate-lab`**



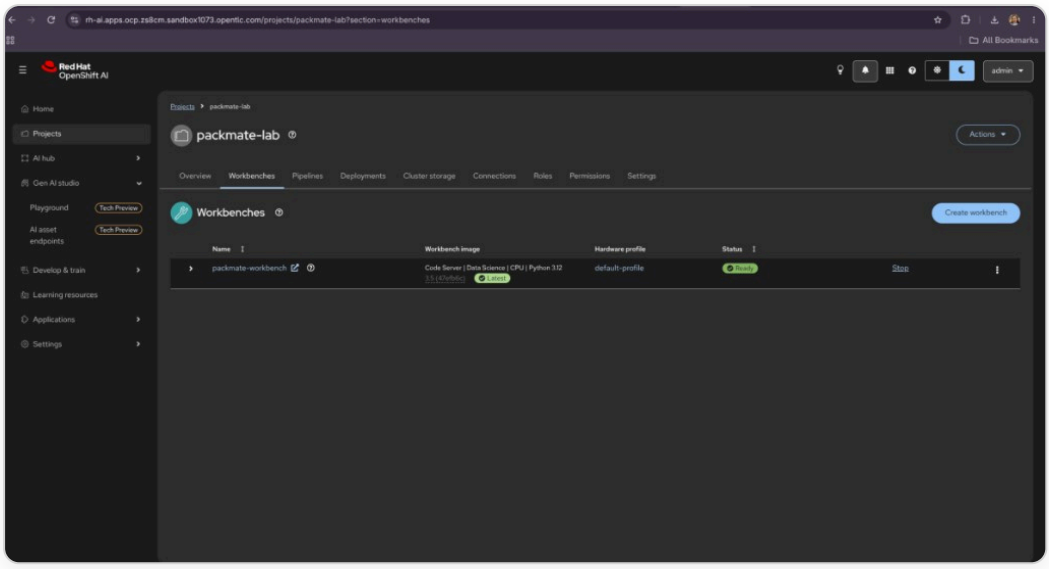
Screenshot: Create project dialog for `packmate-lab`



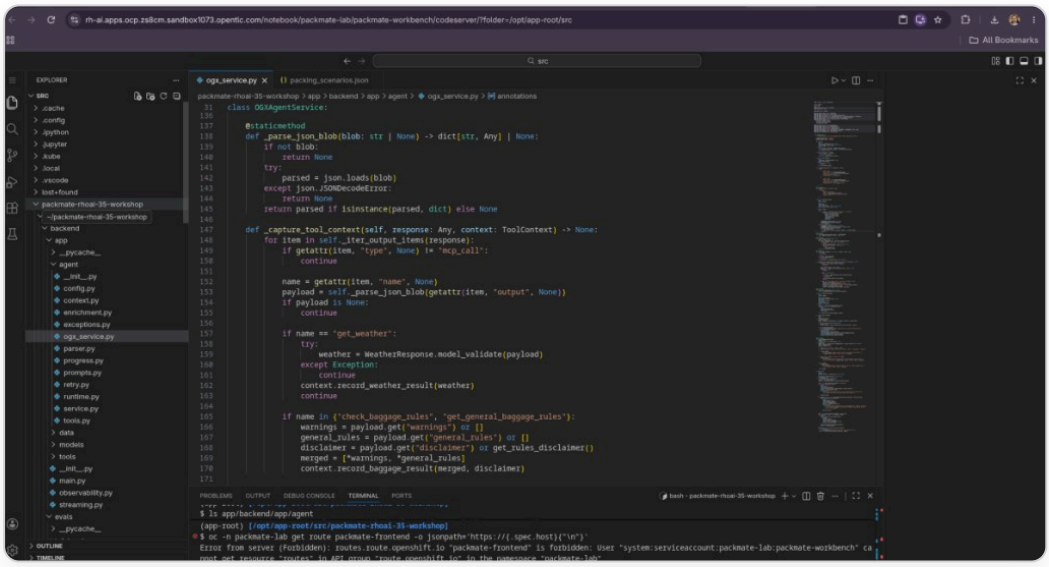
Screenshot: Create Workbench form for `packmate-workbench`



Screenshot: Workbench list showing `packmate-workbench` running



Screenshot: Open Workbench action from the list



EXPECTED RESULT

The Code Server Workbench opens successfully.

WHAT JUST HAPPENED?

You created the browser-based development environment where you will clone the repo and run Python code without needing local tools.

If you are new to OpenShift AI, the key idea is:

- a **project** is your working space
- a **workbench** is your interactive development environment
- **Code Server** gives you an in-browser editor and terminal

In other words, this is the place where the data scientist writes or inspects application code, not just experiments in a chat UI.

CHECKPOINT

Before continuing, make sure `packmate-lab` exists, `packmate-workbench` is `Running`, and the Workbench opens in your browser.

IF IT DOESN'T WORK

See `workshop/TROUBLESHOOTING.md` for Workbench and PVC issues.

Module 2 - Bootstrap the workshop environment

WHAT YOU WILL DO

Clone the workshop repository, authenticate the OpenShift CLI as yourself, and prepare the workshop environment in your sandbox.

WHY THIS MATTERS

This workshop is self-service. You prepare the workshop services once, then you consume them as a data scientist or AI developer.

TIME

10 to 15 minutes

STEP-BY-STEP INSTRUCTIONS

1. In Code Server, open a terminal.
2. Clone the workshop repository:

```
git clone https://github.com/Lindagh1/packmate-rhoai-35-workshop.git
cd packmate-rhoai-35-workshop
```

3. In the terminal, check your current CLI identity:

```
oc whoami
```

4. If the result starts with `system:serviceaccount:`, log in with your human sandbox credentials by pasting the `oc login ...` command that you copied from the OpenShift console into the terminal.
5. Run the one-time workshop bootstrap:

```
make workshop-ready
```

6. Wait until the command completes successfully.

EXPECTED RESULT

The workshop services, MCP integration, application route, OGX connection, and project pipeline server are prepared for the remaining modules.

WHAT JUST HAPPENED?

You performed the one-time environment bootstrap for your own sandbox.

This bootstrap prepares:

- the Packmate application in `packmate-lab`

- the Weather and Baggage MCP servers
- the MCP registration used by Playground
- the OGX-connected backend configuration
- the project pipeline server used later for evaluation

After this step, the rest of the workshop is about using and understanding the prepared AI workflow.

CHECKPOINT

Before continuing, make sure the repository is cloned, `oc whoami` returns your user account, and `make workshop-ready` completed successfully.

IF IT DOESN'T WORK

Run:

```
make preflight
make prepare-workshop
make verify-workshop
```

If the failure persists, ask the instructor for help.

Module 3 - Meet the shared model

WHAT YOU WILL DO

Inspect the shared Llama model that the platform team already deployed.

WHY THIS MATTERS

AI developers often reuse a platform-provided model instead of deploying their own.

TIME

15 minutes

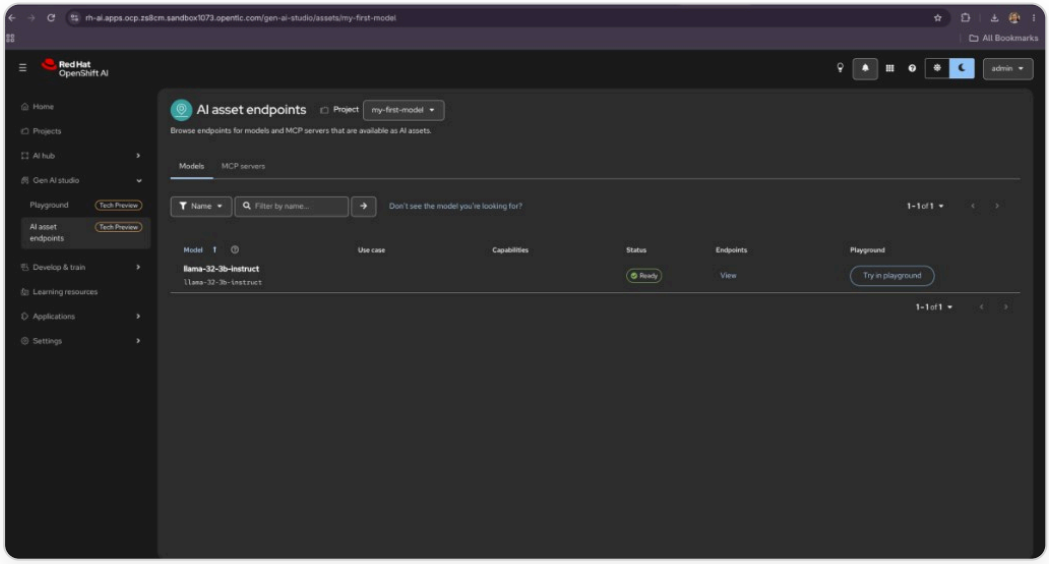
STEP-BY-STEP INSTRUCTIONS

1. In the left navigation, select **Gen AI studio**.
2. Select **AI asset endpoints**.
3. In **Project**, choose `my-first-model` if it is not already selected.
4. Confirm that the shared model `llama-32-3b-instruct` is available.
5. In the Workbench terminal, run:

```
python examples/01_call_model.py
```

SCREENSHOTS

> **Screenshot: AI asset endpoints page with the shared model visible**



EXPECTED RESULT

You see a model response in the terminal.

WHAT JUST HAPPENED?

You reused the shared OpenShift AI model endpoint without deploying another model.

This is an important platform habit to learn:

- you do not always deploy your own model
- sometimes the platform team exposes a shared model for many users
- as a data scientist, you mainly need to know where to find it and how to call it

In this workshop, `my-first-model` is the project that exposes the shared model.

CHECKPOINT

Before continuing, make sure you can see `llama-32-3b-instruct` in `my-first-model` and that the Python example returns a response in the Workbench terminal.

IF IT DOESN'T WORK

Confirm that the shared model in `my-first-model` is `Ready`.

Module 4 - Prototype in Playground

WHAT YOU WILL DO

Chat with the shared model in Playground and add Packmate system instructions.

WHY THIS MATTERS

Playground is the fastest place to experiment before you write application code.

TIME

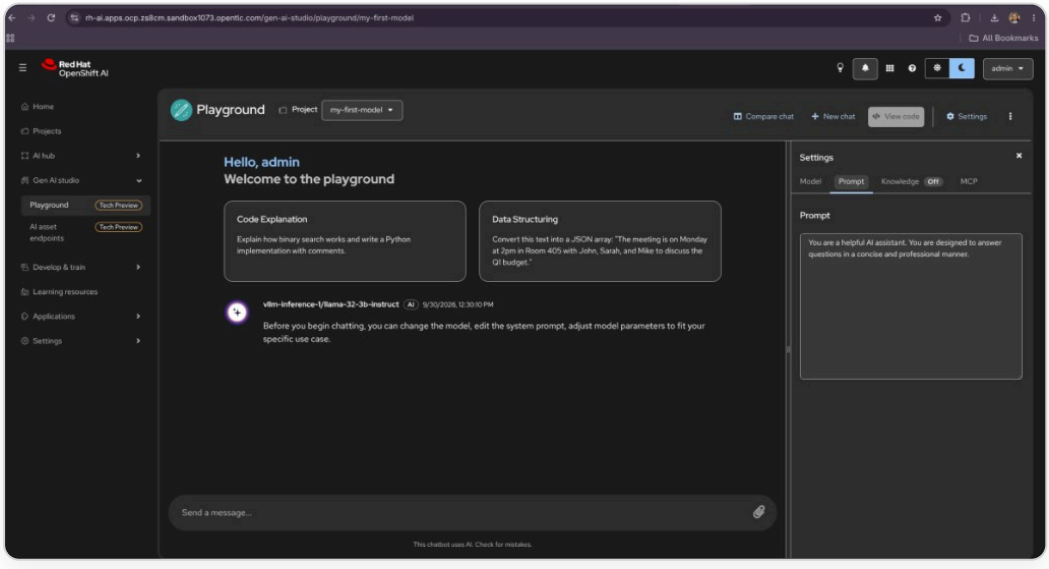
20 minutes

STEP-BY-STEP INSTRUCTIONS

1. In the left navigation, select **Gen AI studio**.
2. Select **Playground**.
3. In **Project**, choose `my-first-model`.
4. If needed, click **Create playground**.
5. Select the `llama-32-3b-instruct` model.
6. Ask: `I am going to Rome for four days. What should I pack?`
7. Open the **Prompt** tab.
8. Paste the contents of `workshop/assets/system-instructions.md`.
9. Ask the same question again and compare the result.

SCREENSHOTS

> **Screenshot: Playground with the shared model selected and the Prompt tab open**



EXPECTED RESULT

The second answer behaves more like Packmate.

WHAT JUST HAPPENED?

System instructions changed how the assistant behaves without retraining the model.

This is one of the most important beginner lessons:

- the **model** stayed the same
- the **system prompt** changed the behavior
- this is often the first step before writing application code

Playground is useful because it lets you learn prompt behavior quickly before you commit anything to the application.

CHECKPOINT

Before continuing, make sure the model responds in Playground and that you can explain the difference between a user question and the Packmate system instructions.

IF IT DOESN'T WORK

Re-open the Playground and confirm the selected project and model.

Module 5 - Give Packmate tools with MCP

WHAT YOU WILL DO

Enable the prepared Weather and Baggage Policy MCP servers in Playground and inspect how tool calls work.

WHY THIS MATTERS

MCP gives the model live capabilities instead of only static knowledge.

TIME

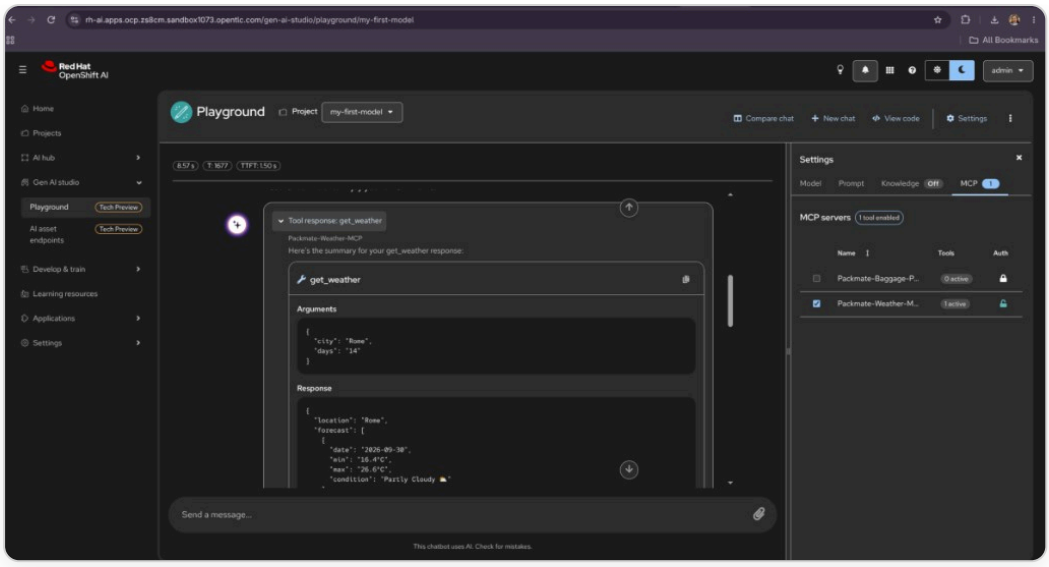
20 to 25 minutes

STEP-BY-STEP INSTRUCTIONS

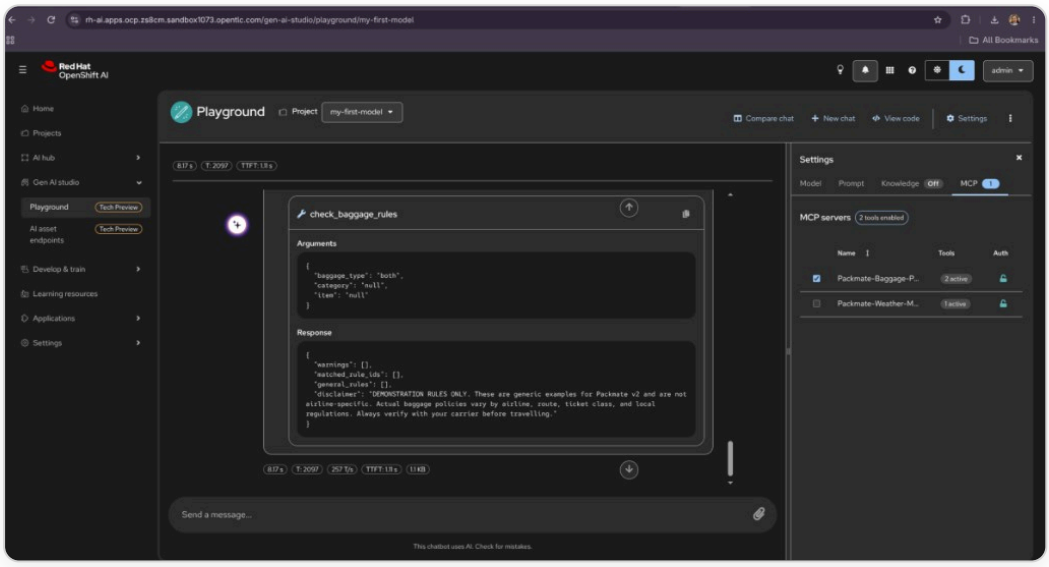
1. In Playground, open the **MCP** tab.
2. Enable the Packmate Weather MCP server.
3. Ask: `I am travelling to Rome next week. Check the weather and recommend what I should pack.`
4. Inspect the tool call details. Notice: - which tool the model selected - which arguments were sent - what the tool returned
5. Enable the Packmate Baggage Policy MCP server.
6. Ask: `Can I take a 150 ml liquid in cabin baggage?`
7. Ask: `Can I put my power bank in checked baggage?`
8. Inspect the tool calls.
9. Optionally disable one MCP server and retry a related question to see how the assistant falls back when a tool is unavailable.

SCREENSHOTS

> **Screenshot: Playground MCP panel with the Weather server enabled**



Screenshot: Playground MCP panel with the Baggage Policy server enabled



EXPECTED RESULT

You see the tools being invoked and then used in the final answer.

WHAT JUST HAPPENED?

MCP stands for Model Context Protocol. In this workshop:

- an MCP server exposes one or more tools
- the Playground sends the model the tool definitions
- the model decides whether it needs a tool
- the tool runs outside the model
- the tool result is returned to the model
- the model uses that result in its final answer

When you open the tool details, try to read them like an AI developer:

- what question did I ask?
- which tool did the model choose?
- what arguments did it send?
- what raw result came back?
- how did the final answer change because of that result?

That is the core mental model of agentic AI in this workshop.

CHECKPOINT

Before continuing, make sure both MCP servers are enabled and that you can explain which tool the model chose, which arguments it sent, and how the tool result changed the final answer.

IF IT DOESN'T WORK

Ask the instructor to run `make verify-workshop`.

Module 6 - From Playground to Code Server and Python

WHAT YOU WILL DO

Open the Packmate repository in Code Server and run the Python examples from the Workbench terminal.

WHY THIS MATTERS

This is the bridge from experimentation to application code.

HOW TO THINK ABOUT THIS STEP

At this point, you switch from "Playground user" to "AI developer".

In this workshop, Code Server is where you inspect and understand the project:

- `app/frontend` contains the React frontend
- `app/backend` contains the FastAPI backend
- `mcp/weather` contains the Weather MCP server
- `mcp/baggage` contains the Baggage Policy MCP server
- `examples/01_call_model.py` calls the shared model directly
- `examples/02_packmate_with_tools.py` calls the deployed Packmate application
- `examples/03_evaluate_packmate.py` runs the deterministic evaluation

HOW PACKMATE USES OGX

This workshop now follows the OpenShift AI `3.5` agentic path more closely:

Keep this mental model in mind:

- the platform team prepared OGX for you
- the shared model is still served by `KServe + ServingRuntime + vLLM`
- the Packmate application path is `React frontend -> FastAPI backend -> OGX`
- OGX is the runtime that connects the model to the prepared MCP servers
- you do **not** administer OGX in this workshop

Why this matters:

- Playground shows the behavior from the UI
- Code Server shows where that behavior lives in code
- the deployed app uses the same model and the same MCP tools through OGX
- the repository shows you which component owns which part of the flow

If you want to inspect the code path, look at:

- `app/backend/app/main.py`

- `app/backend/app/agent/runtime.py`
- `app/backend/app/agent/ogx_service.py`

TIME

15 minutes

STEP-BY-STEP INSTRUCTIONS

1. Open the terminal in Code Server.
2. In the Code Server file explorer, open the cloned repository and inspect:

- `app/frontend`
- `app/backend`
- `mcp/weather`
- `mcp/baggage`

Also inspect these files if you want to understand the runtime path:

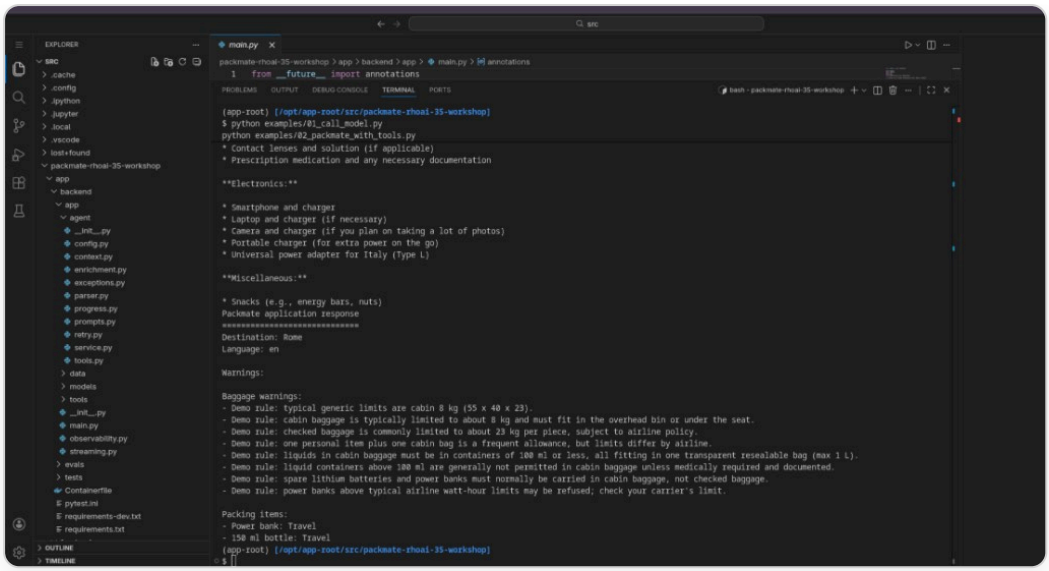
- `app/backend/app/agent/ogx_service.py`
- `app/backend/app/agent/prompts.py`
- `mcp/weather/src/weather_mcp/app.py`
- `mcp/baggage/src/baggage_policy_mcp/app.py`
- `scripts/prepare-workshop.sh`
- `examples/01_call_model.py`
- `examples/02_packmate_with_tools.py`

3. In the Code Server terminal, run:

```
python examples/01_call_model.py
python examples/02_packmate_with_tools.py
```

SCREENSHOT

> **Screenshot: Workbench terminal running the Packmate Python examples**



EXPECTED RESULT

The first script calls the shared model directly so that you can see the base model behavior.

The second script calls the deployed Packmate application. Inside the application:

- the React frontend talks to the FastAPI backend
- the FastAPI backend sends the request to OGX
- OGX uses the shared model plus the prepared MCP servers
- the final structured answer comes back to Packmate

WHAT JUST HAPPENED?

You moved from UI experimentation to readable application code.

As a data scientist or AI developer, this is the key mental model:

- Playground helps you prototype prompts and tool behavior
- Code Server is where you inspect and evolve the application
- the frontend is a React UI
- the backend is a FastAPI service
- the MCP servers are separate tool services
- the shared model endpoint is reused instead of deployed by the participant
- the backend uses OGX as the standard agentic runtime path
- OGX does not replace `KServe` or `vLLM`; it sits in front of them for this workflow
- the Weather and Baggage MCP servers are still normal tool services that OGX can call

Use this short map when you inspect the repository:

- `app/frontend`: the React user interface
- `app/backend`: the FastAPI application API

- `app/backend/app/agent/ogx_service.py` : the OGX-backed AI orchestration
- `mcp/weather` and `mcp/baggage` : the two MCP tool servers
- `scripts/prepare-workshop.sh` : the workshop bootstrap that discovers the OGX endpoint and configures the backend
- `pipelines/packmate-evaluation.pipeline.yaml` : the evaluation pipeline definition

CHECKPOINT

Before continuing, make sure you can run both Python examples and point to the frontend, backend, MCP servers, and OGX-backed runtime path in the repository.

IF IT DOESN'T WORK

Verify that the Workbench can reach in-cluster services and that the workshop preparation completed successfully.

Module 7 - Open the integrated Packmate application

WHAT YOU WILL DO

Open the deployed Packmate web application and compare it with Playground behavior.

WHY THIS MATTERS

This shows how prototype behavior becomes part of an application with extra controls.

TIME

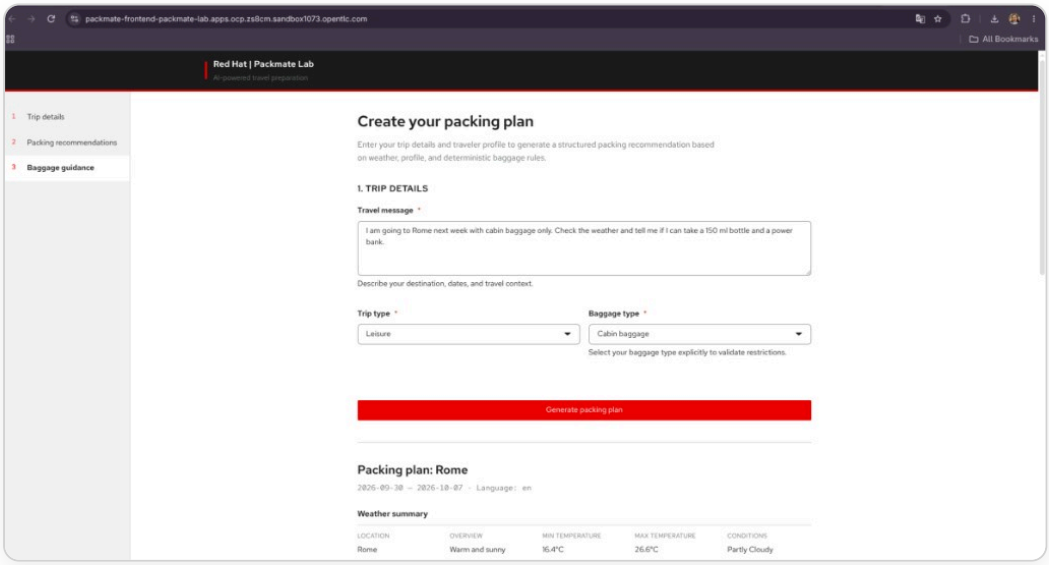
15 minutes

STEP-BY-STEP INSTRUCTIONS

1. Open the Packmate Route provided by the instructor.
2. Submit the same scenario that you used in Playground.
3. Compare the result.

SCREENSHOT

Screenshot: Packmate web application answering a travel question



EXPECTED RESULT

The application returns a structured response.

WHAT JUST HAPPENED?

The frontend called the Packmate backend, and the backend sent an OGX request that used the shared model plus MCP-backed capabilities.

This is the final beginner connection to make:

- in Playground, you manually explore the behavior
- in the app, the same mechanism is wrapped in a product flow
- the user does not need to know MCP or OGX internals
- the AI developer does need to understand how those layers connect

Important:

- the app you opened is the deployed in-cluster workshop application
- you are not launching the main Packmate web app locally in this module
- the local repository helps you understand the mechanism behind the deployed app

CHECKPOINT

Before continuing, make sure you opened the deployed Packmate application and received a structured answer for a travel scenario.

IF IT DOESN'T WORK

Ask the instructor to run `make diagnose`.

Module 8 - Evaluate the application

WHAT YOU WILL DO

Run the deterministic Packmate regression evaluation from OpenShift AI Pipelines.

WHY THIS MATTERS

Evaluation helps you detect regressions in application behavior and turn the workshop check into a repeatable project workflow.

TIME

10 to 15 minutes

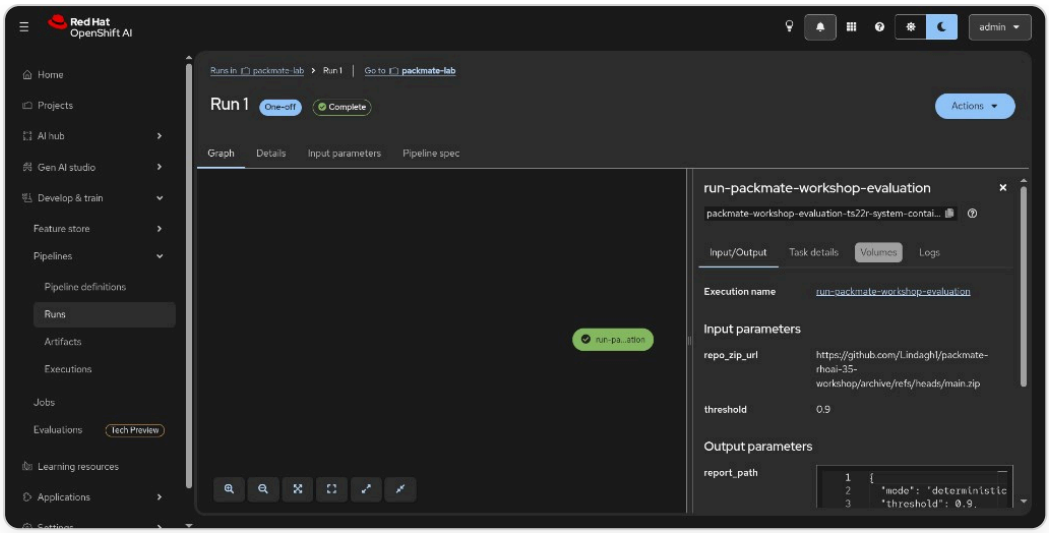
STEP-BY-STEP INSTRUCTIONS

1. In Code Server, open `pipelines/packmate-evaluation.pipeline.yaml`.
2. Download that YAML file from Code Server to your machine.
3. In OpenShift AI, return to the `packmate-lab` project.
4. Open the **Pipelines** tab.
5. Confirm that the project pipeline server is already available.
6. Upload `pipelines/packmate-evaluation.pipeline.yaml`.
7. Create a pipeline run with the default parameters.
8. Wait for the run to complete.
9. Open the run details and inspect the logs or report output.
10. Optionally, in the Code Server terminal, inspect the equivalent local script:

```
python examples/03_evaluate_packmate.py
```

SCREENSHOT

> **Screenshot: Pipeline run result for the Packmate evaluation**



EXPECTED RESULT

You get a PASS/FAIL score for the deterministic workshop regression checks.

WHAT JUST HAPPENED?

The pipeline executed the workshop evaluation as a project artifact instead of a one-off terminal command.

This is useful because:

- a project pipeline is easier to rerun
- the result is attached to a named pipeline run
- it is closer to how teams operationalize evaluations over time

The underlying evaluation logic is still the same Packmate regression check. It is not a claim that the model is universally accurate.

Important:

- the instructor automation prepares the project pipeline server for you
- you should not have to enter S3 or MinIO credentials manually in the participant path

CHECKPOINT

Before continuing, make sure you uploaded the pipeline YAML, launched the run, and understand that this evaluation is a repeatable regression check rather than a universal accuracy score.

IF IT DOESN'T WORK

For local troubleshooting, you can still run the equivalent script in Code Server:

```
python examples/03_evaluate_packmate.py
```

Module 9 - Understand the architecture

WHAT YOU WILL DO

Review the beginner-level architecture.

WHY THIS MATTERS

You should leave the workshop understanding what OpenShift AI provided for you.

TIME

10 minutes

STEP-BY-STEP INSTRUCTIONS

1. Open `workshop/ARCHITECTURE.md`.
2. Review the participant flow, serving architecture, and MCP sequence.

EXPECTED RESULT

You can explain the role of Workbench, Playground, OGX, KServe, vLLM, MCP, and evaluation.

CHECKPOINT

Before continuing, make sure you can explain the role of the Workbench, Playground, OGX, KServe, vLLM, MCP, and the Packmate application in one short flow.

Module 10 - From prototype to production

WHAT YOU WILL DO

Review the short productionization overview.

WHY THIS MATTERS

A useful prototype still needs more work before production.

TIME

5 minutes

STEP-BY-STEP INSTRUCTIONS

1. Review the optional productionization discussion from the instructor.

EXPECTED RESULT

You understand that application packaging, CI/CD, promotion, and operational controls come after prototype validation.